Министерство образования и науки Российской Федерации

Федеральное Государственное Автономное Образовательное

Учреждение Высшего Образования "Национальный

Исследовательский Университет ИТМО"

Факультет программной инженерии и компьютерной техники

Лабораторная работа №3

По дисциплине "Системы ввода-вывода"

Тема: "Изучение протоколов передачи данных между устройствами"

Вариант: SPI

Выполнили студенты группы P3319

Копытов Никита Эдуардович

Лаврентьев Лев Денисович

Волокитин Александр Сергеевич

Проверил преподаватель практики

Быковский Сергей Вячеславович

г. Санкт-Петербург, 2026 г.

Цель работы

Познакомиться с алгоритмами обмена и форматами передачи данных, принципами обмена данными между устройствами на примере интерфейсов SPI, 1-Wire.

Задание

1. Подключить комплект контроллера с датчиком и логическим анализатором к компьютеру;
2. С помощью логического анализатора записать временную диаграмму обмена данными по сигнальным линиям в течение трёх транзакций обмена;
3. Расшифровать протокол обмена данными;
4. Перевести значение физической величины, заданной в варианте задания, в человекочитаемый формат;
5. Нарисовать временную диаграмму передачи другого, отличного от полученных, значения физической величины;
6. Определить скорость интерфейса;
7. Оформить отчёт по работе в электронном формате.

Теоретические сведения

Интерфейс SPI

SPI (Serial Peripheral Interface) - синхронный последовательный интерфейс передачи данных с архитектурой master-slave. Использует четыре сигнальные линии:

- **SCK** (Serial Clock) - тактовая линия, генерируется ведущим (Master);
- **MOSI** (Master Out - Slave In) - данные от ведущего к ведомому;
- **MISO** (Master In - Slave Out) - данные от ведомого к ведущему;
- **CS/SS** (Chip Select) - линия выбора ведомого устройства (активный низкий уровень).

Режим работы SPI определяется двумя параметрами:

- **CPOL** (Clock Polarity) - полярность сигнала синхронизации в состоянии покоя;
- **CPHA** (Clock Phase) - фаза сигнала синхронизации, определяющая, по какому фронту защёлкиваются данные.

В данной работе используется **SPI Mode 0** (CPOL = 0, CPHA = 0), так как именно этот режим применяется микроконтроллером Arduino Nano при работе с датчиком BMP280.

Датчик BMP280

BMP280 - цифровой барометрический датчик давления со встроенным термометром производства Bosch Sensortec. Поддерживает интерфейсы I²C и SPI (3-wire и 4-wire), работает на частоте SPI до 10 МГц.

Фрагмент карты регистров BMP280 (источник - datasheet BST-BMP280-DS001-11, раздел 4.2):

Адрес	Регистр	Назначение
0x88 – 0x8D	calib00–calib05	Калибровочные коэффициенты температуры
0x8E – 0x9F	calib06–calib23	Калибровочные коэффициенты давления
0xD0	id	Идентификатор чипа (должен быть 0x58)

0xF7 – 0xF9	press_msb, lsb, xlsb	Сырое значение давления (20 бит)
0xFA – 0xFC	temp_msb, lsb, xlsb	Сырое значение температуры (20 бит)

Адресация при SPI-чтении BMP280

Согласно разделу 5.3.2 datasheet BMP280, первый байт транзакции - управляющий:

- Старший бит (бит 7) = RW: **1 - чтение**, **0 - запись**;
- Биты 6…0 - адрес регистра.

Для чтения регистра с адресом `0xFA` (старший бит адреса уже равен 1) ведущий передаёт байт `0xFA` напрямую. После этого BMP280 автоматически инкрементирует адрес (auto-increment), позволяя одной командой прочитать несколько регистров подряд.

Схема подключения

Подключение BMP280 к Arduino Nano по 4-проводному SPI:

Arduino Nano	BMP280	Сигнал SPI
D13	SCL	SCK
D11	SDA (SDI)	MOSI
D12	SDO	MISO
D10	CSB	CS
3.3 В	VCC	Питание
GND	GND	Общая земля

Логический анализатор Saleae Logic (клон, 24 МГц, 8 каналов) подключён параллельно к сигнальным линиям SPI между Arduino Nano и BMP280:

Канал анализатора	Сигнал
D2	MISO
D3	MOSI
D4	CS
D5	SCK
GND	Общая земля

Результаты измерений

Запись в Logic 2

Запись обмена по SPI осуществлялась в ПО Saleae Logic 2 на частоте дискретизации 24 MS/s в течение приблизительно 15 секунд. Микроконтроллер Arduino Nano в это время выполнял инициализацию BMP280 и периодическое чтение температуры.

Настройки SPI-анализатора в Logic 2:

Параметр	Значение
MOSI	Channel 3
MISO	Channel 2
Clock	Channel 5
Enable	Channel 4
Significant Bit	MSB First (Standard)
Bits per Transfer	8 Bits (Standard)
Clock State	CPOL = 0
Clock Phase	CPHA = 0
Enable Line	Active Low (Standard)

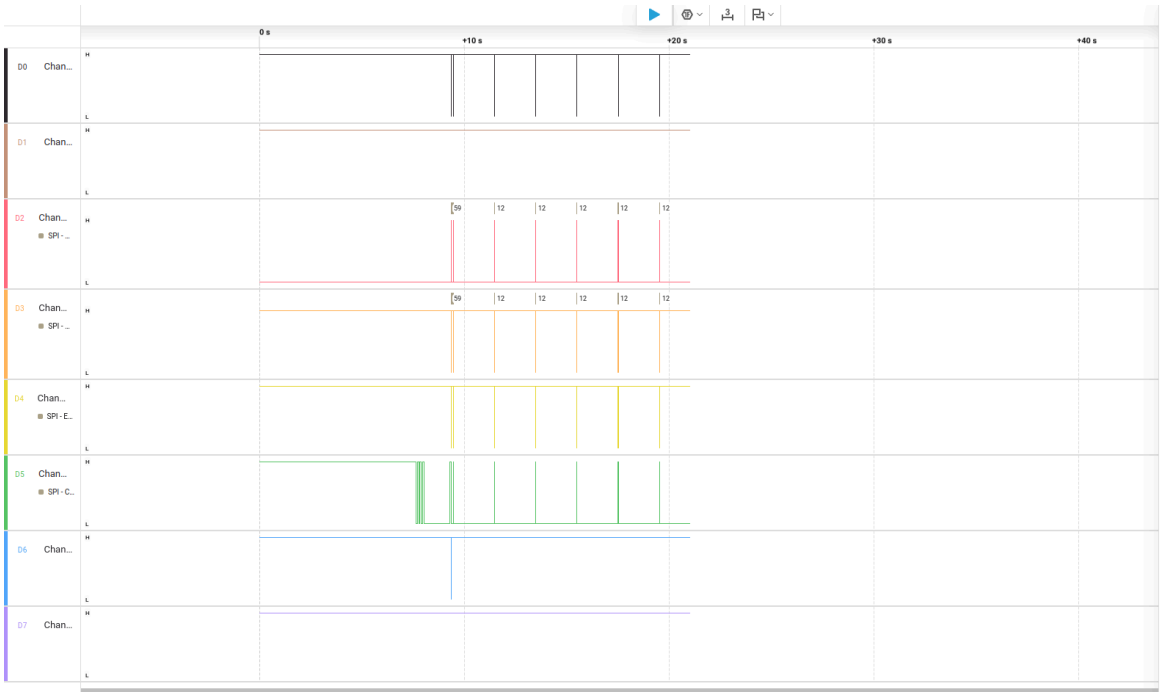

Рис. 1. Общий вид записи в Logic 2: видны периодические транзакции обмена по SPI.

Чтение ID датчика

При инициализации Arduino Nano считывает регистр `0xD0` для проверки наличия датчика:

MOSI	MISO	Описание
0xD0	0x00	Команда: чтение регистра 0xD0 (id)
0xFF	0x58	Ответ датчика: 0x58 - ID BMP280

Полученное значение `0x58` соответствует идентификатору BMP280 согласно разделу 4.3.1 datasheet. Датчик исправен и корректно отвечает на запросы.

Чтение калибровочных коэффициентов температуры

Сразу после чтения ID Arduino Nano считывает из NVM-памяти BMP280 калибровочные коэффициенты `dig_T1`, `dig_T2`, `dig_T3` из регистров `0x88`…`0x8D`. Эти коэффициенты уникальны для каждого экземпляра датчика и прошиваются на заводе.

Согласно таблице 17 datasheet, в BMP280 байты коэффициентов располагаются в порядке **LSB, MSB** (little-endian). Значения, зафиксированные в ходе работы:

Коэфф.	Регистры	MISO (LSB, MSB)	Тип	Значение
dig_T1	0x88 / 0x89	0xEE, 0x6B	unsigned 16	27630
dig_T2	0x8A / 0x8B	0xDC, 0x67	signed 16	26588
dig_T3	0x8C / 0x8D	0x18, 0xFC	signed 16	−1000

Пояснение к преобразованию:

- **dig_T1:** `(0x6B << 8) | 0xEE = 0x6BEE = 27630` (беззнаковое).
- **dig_T2:** `(0x67 << 8) | 0xDC = 0x67DC = 26588` (старший бит 0 ⟶ положительное).
- **dig_T3:** `(0xFC << 8) | 0x18 = 0xFC18 = 64536`. Так как старший бит = 1, число отрицательное (two's complement): `64536 – 65536 = -1000`.

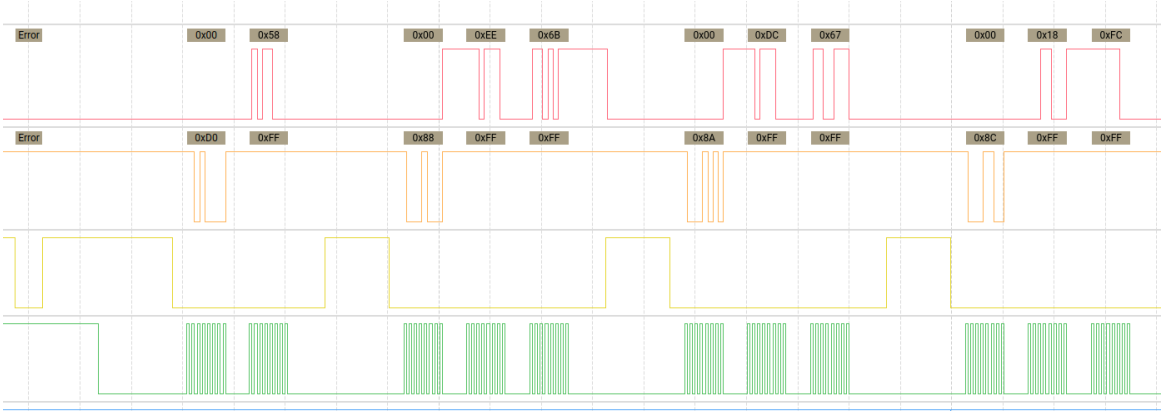

Рис. 2. Транзакции чтения калибровочных коэффициентов температуры.

Чтение температуры (три транзакции)

Каждая транзакция чтения температуры состоит из **4 байт**: один байт команды и три байта ответа.

Структура транзакции:

#	MOSI	MISO	Описание
1	0xFA	0x00	Команда: чтение регистра 0xFA (temp_msb)
2	0xFF	temp_msb	Ответ: старший байт температуры
3	0xFF	temp_lsb	Ответ: средний байт температуры
4	0xFF	temp_xlsb	Ответ: младший байт (старшие 4 бита)

Байты MOSI `0xFF` - это dummy-байты: Arduino продолжает тактировать SCK, чтобы получить от BMP280 оставшиеся байты ответа. Благодаря механизму auto-increment, адрес регистра последовательно увеличивается: `0xFA → 0xFB → 0xFC`.

Результаты трёх последовательных транзакций:

№	MOSI	MISO	temp_msb, lsb, xlsb	adc_T
1	FA FF FF FF	00 7F B6 80	0x7F, 0xB6, 0x80	0x7FB68 = 523112
2	FA FF FF FF	00 7F B9 80	0x7F, 0xB9, 0x80	0x7FB98 = 523160
3	FA FF FF FF	00 7D D8 80	0x7D, 0xD8, 0x80	0x7DD88 = 515976

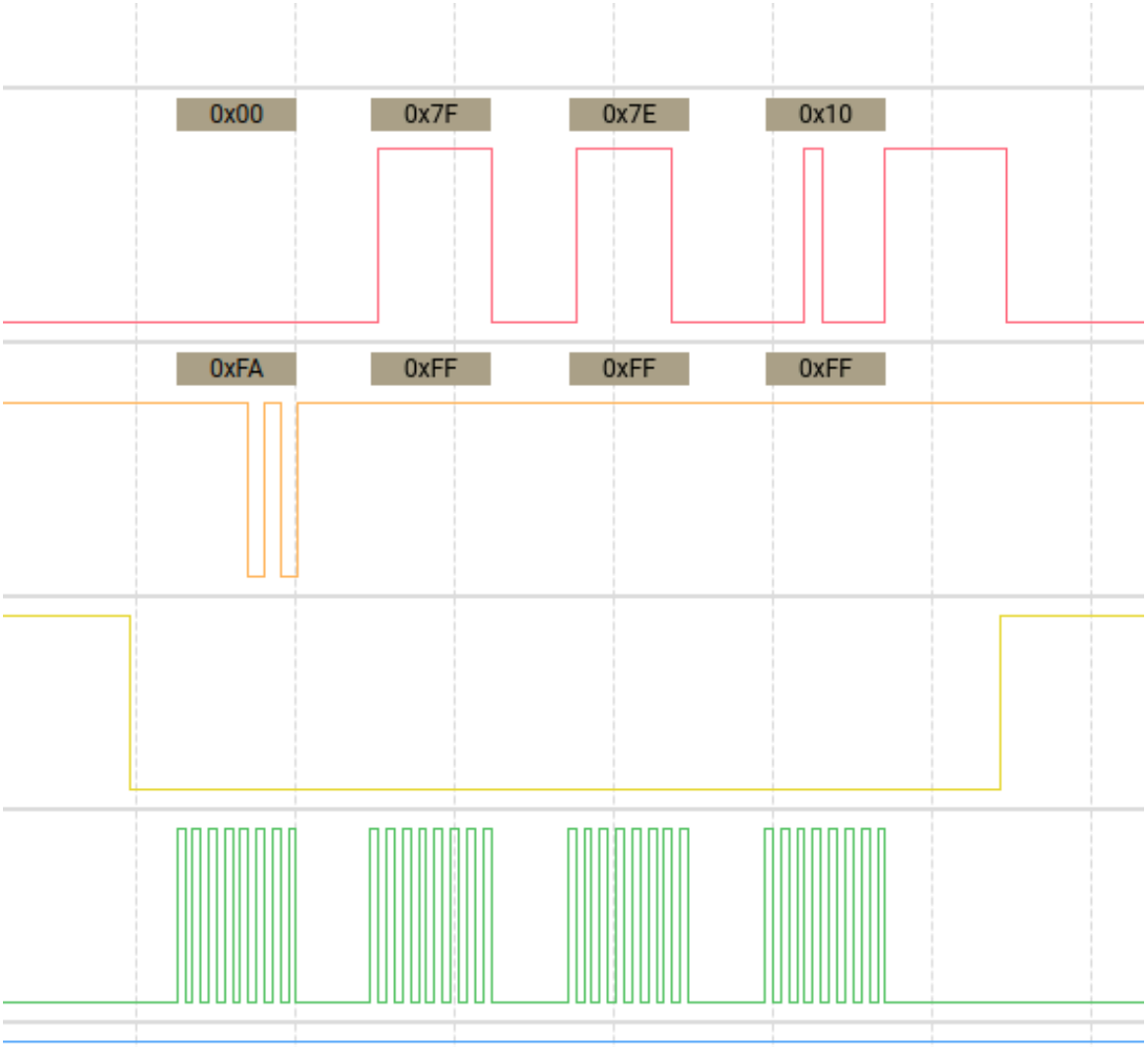

Рис. 3. Временная диаграмма одной транзакции чтения температуры.

Перевод температуры в градусы Цельсия

Сборка 20-битного значения adc_T

Согласно разделу 4.3.7 datasheet, сырое значение температуры формируется из трёх байтов (`temp_msb`, `temp_lsb`, `temp_xlsb`). Младшие 4 бита регистра `0xFC` (temp_xlsb) не используются:

$$\text{adc_T} = (\text{temp_msb} \ll 12) \vee (\text{temp_lsb} \ll 4) \vee (\text{temp_xlsb} \gg 4)$$

Транзакция №1 (`0x7F, 0xB6, 0x80`): `adc_T = 0x7FB68 = 523112`.

Формула компенсации

Перевод adc_T в градусы Цельсия выполняется по формуле из раздела 3.11.3 / 8.1 datasheet (с плавающей точкой):

$$\text{var1} = \left(\frac{\text{adc_T}}{16384} - \frac{\text{dig_T1}}{1024}\right) \cdot \text{dig_T2}$$

$$\text{var2} = \left(\left(\frac{\text{adc_T}}{131072} - \frac{\text{dig_T1}}{8192}\right)^2\right) \cdot \text{dig_T3}$$

$$T = \frac{\text{var1} + \text{var2}}{5120}$$

Формула учитывает заводскую калибровку индивидуального датчика через коэффициенты `dig_T1` (смещение), `dig_T2` (линейная компенсация) и `dig_T3` (квадратичная компенсация нелинейности).

Расчёт для транзакции №1

С калибровкой (`dig_T1 = 27630`, `dig_T2 = 26588`, `dig_T3 = -1000`) и `adc_T = 523112`:

$$\text{var1} \approx (31.9287 - 26.9824) \cdot 26588 \approx 131517$$

$$\text{var2} \approx (3.9910 - 3.3728)^2 \cdot (-1000) \approx -382.2$$

$$T = \frac{131517 - 382.2}{5120} \approx 25.6°C$$

Результаты для всех трёх транзакций

№	adc_T	var1	var2	T, °C
1	523112	131517	−382.2	25.6
2	523160	131772	−382.5	25.66
3	515976	113725	−347.9	22.14

Временная диаграмма передачи значения 30.0 °C

По заданию требуется построить временную диаграмму передачи температуры, отличной от измеренной. Выполним **обратный расчёт**: найдём `adc_T`, соответствующее **T = 30.0 °C**, с использованием полученных калибровочных коэффициентов.

Обратный расчёт adc_T для T = 30.0 °C

Численным подбором (1 °C ≈ 2040 единиц adc_T) получаем `adc_T ≈ 0x82C4C = 535500`.

Проверка: `var1 ≈ 155140`, `var2 ≈ -496`, `T ≈ 30.2 °C`.

Итоговое значение: `adc_T = 0x82C4C`.

Разложение на три байта:

- temp_msb = биты [19:12] = `0x82`
- temp_lsb = биты [11:4] = `0xC4`
- temp_xlsb = биты [3:0], записанные в старших 4 битах регистра = `0xC0`

Ожидаемые значения на линиях

При передаче T ≈ 30 °C транзакция содержала бы следующие байты:

#	MOSI	MISO	Описание
1	0xFA	0x00	Команда: чтение temp_msb
2	0xFF	0x82	temp_msb
3	0xFF	0xC4	temp_lsb
4	0xFF	0xC0	temp_xlsb (значимые 4 бита)

Временная диаграмма

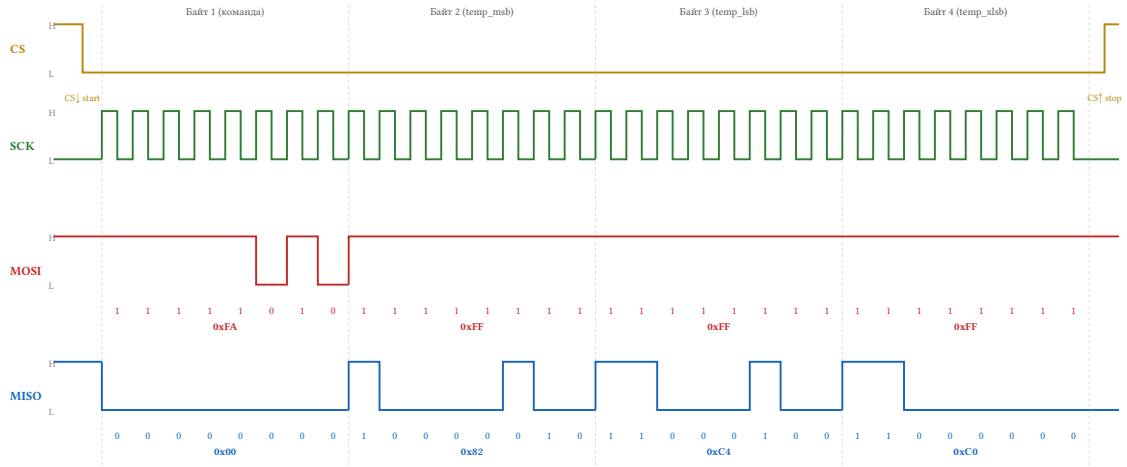

Рис. 4. Временная диаграмма передачи T ≈ 30.0 °C по SPI.

На диаграмме показаны:

- CS переходит в LOW - начало транзакции;
- На SCK генерируется 32 такта (4 байта по 8 бит);
- На MOSI последовательно выводятся байты `0xFA, 0xFF, 0xFF, 0xFF`;
- На MISO датчик выводит байты `0x00, 0x82, 0xC4, 0xC0`;
- CS возвращается в HIGH - конец транзакции.

Определение скорости интерфейса

По записи в Logic 2 определена длительность передачи одного байта по SPI. Один байт состоит из 8 тактовых циклов SCK.

Из измерений в программе Logic 2:

$$T_{\text{byte}} = 7.583 \text{ мкс}$$

Тогда частота SCK:

$$f_{\text{SCK}} = \frac{8}{T_{\text{byte}}} = \frac{8}{7.583 \cdot 10^{-6}} \approx 1.055 \text{ МГц}$$

Скорость интерфейса SPI: ≈ 1 МГц.

Это типичная скорость для Arduino Nano, использующего стандартный делитель системной тактовой частоты 16 МГц на 16. Данная скорость находится в пределах допустимой для BMP280 (до 10 МГц согласно datasheet).

Исследование датчика DHT-11 по интерфейсу single-wire

Описание интерфейса

DHT-11 использует собственный однопроводной протокол компании Aosong. Передача ведётся по одной двунаправленной линии `DATA`, подтянутой внешним резистором ≈ 5,1 кОм к питанию. Оба устройства имеют выходы с открытым стоком: в состоянии покоя линия удерживается в `HIGH`, а для передачи ведущий или ведомый кратковременно притягивает её к `GND`.

Обмен всегда инициирует микроконтроллер. DHT-11 — пассивный датчик: он находится в режиме пониженного энергопотребления и отвечает только после сигнала запуска от мастера.

Формат транзакции

Каждая транзакция состоит из трёх частей:

1. **Стартовый сигнал мастера.** МК переводит линию в `LOW` на время не менее 18 мс, затем отпускает её. Подтяжка возвращает линию в `HIGH`.
2. **Отклик датчика.** DHT-11 подтверждает запрос коротким `LOW` ≈ 80 мкс, затем `HIGH` ≈ 80 мкс, после чего начинает передачу данных.
3. **40 бит данных.** Каждый бит начинается с `LOW` ≈ 50 мкс; длительность последующего `HIGH` кодирует значение:
    - `HIGH` ≈ 26–28 мкс — бит “0”;
    - `HIGH` ≈ 70 мкс — бит “1”.

Порядок передачи — старший бит вперёд (MSB first). 40 бит разбиваются на пять байт:

Байт	Назначение
1	Целая часть относительной влажности, %
2	Дробная часть влажности (у DHT-11 всегда 0)
3	Целая часть температуры, °C
4	Дробная часть температуры, десятые доли
5	Контрольная сумма

Table 1: Формат кадра данных DHT-11

Контрольная сумма вычисляется как сумма первых четырёх байт, усечённая до 8 младших бит:

$$\text{CRC} = (\text{байт}_1 + \text{байт}_2 + \text{байт}_3 + \text{байт}_4)\ \text{AND}\ \text{0xFF}$$

При несовпадении CRC пакет следует отбросить и повторить запрос не ранее чем через 2 с (минимальный период опроса по даташиту).

Полученная временная диаграмма

Обмен зафиксирован логическим анализатором Saleae Logic на частоте дискретизации 24 МГц.

Схема подключения: Arduino Nano выступает мастером и периодически опрашивает DHT-11; анализатор подключён параллельно линии `DATA` и работает только на приём, не влияя на обмен.

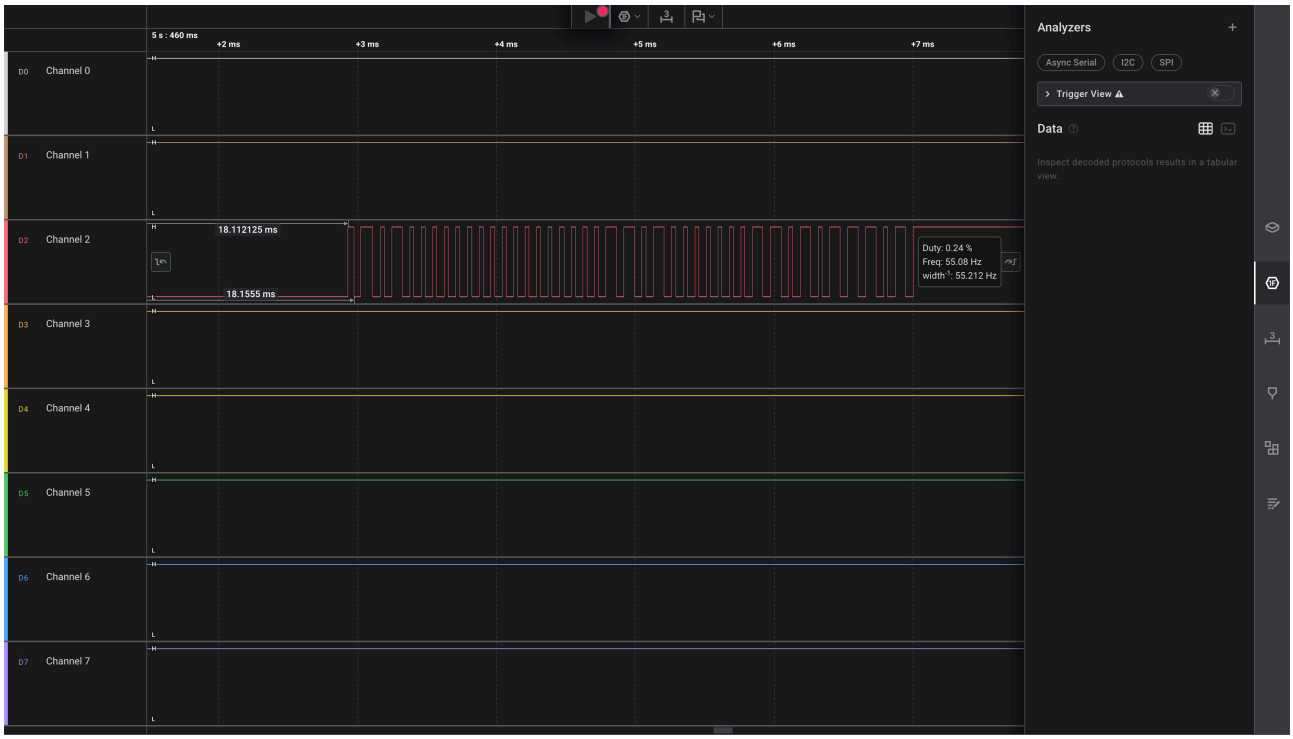

Figure 1: Временная диаграмма одной транзакции DHT-11

За время наблюдения (≈ 8,3 с) зафиксировано 4 транзакции опроса с интервалом ≈ 2 с. Измеренные параметры служебных импульсов приведены в таблице ниже.

Фаза	**По даташиту**	**Измерено**
Старт мастера, `LOW`	≥ 18 мс	18,11 мс
Пауза мастера, `HIGH`	20–40 мкс	43,3 мкс
Отклик датчика, `LOW`	≈ 80 мкс	46,8 мкс
Готовность датчика, `HIGH`	≈ 80 мкс	90,2 мкс
Начало бита, `LOW`	≈ 50 мкс	54,3 мкс
Бит “0”, `HIGH`	26–28 мкс	28,3 мкс
Бит “1”, `HIGH`	≈ 70 мкс	75,3 мкс

Table 2: Измеренные длительности импульсов

Все измеренные значения лежат в допусках, заявленных производителем.

Расшифровка протокола обмена

Для детального разбора возьмём транзакцию №1 (момент времени t ≈ 1,379 с). После стартового импульса и отклика датчика принято 40 бит, которые разбиваются на 5 байт по 8 бит:

№ байта	**Биты (MSB→LSB)**	**HEX**	**DEC**	**Поле**	**Значение**
1	`00100100`	`0x24`	36	RH int	36 %
2	`00000000`	`0x00`	0	RH dec	0,0 %
3	`00011010`	`0x1A`	26	T int	26 °C
4	`00000000`	`0x00`	0	T dec	0,0 °C
5	`00111110`	`0x3E`	62	CRC	—

Table 3: Расшифровка байтов транзакции №1

Пример расчёта первого байта вручную:

$$0\text{x}24 = 0 \cdot 128 + 0 \cdot 64 + 1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 0 \cdot 1 = 36$$

Проверка контрольной суммы:

$$\text{CRC}_{\text{расч}} = (36 + 0 + 26 + 0) \text{ and } 0\text{xFF} = 62 = 0\text{x}3\text{E}$$

Полученное значение совпадает с пятым байтом — данные корректны.

Перевод байтов в физические величины по формулам:

$$\text{RH} = \text{байт}_1 + \frac{\text{байт}_2}{10} = 36 + \frac{0}{10} = 36,0\%$$

$$T = \text{байт}_3 + \frac{\text{байт}_4}{10} = 26 + \frac{0}{10} = 26,0°C$$

Сводные результаты

В таблице ниже приведены результаты декодирования всех четырёх транзакций из захвата.

№	t, c	Байты (hex)	RH, %	T, °C	CRC
1	1,379	`24 00 1A 00 3E`	36,0	26,0	OK
2	3,412	`31 00 1A 01 4C`	49,0	26,1	OK
3	5,445	`40 00 1A 03 5D`	64,0	26,3	OK
4	7,478	`44 00 1A 07 65`	68,0	26,7	OK

Table 4: Результаты декодирования всех транзакций

Заметна выраженная динамика показаний: относительная влажность возросла с 36 % до 68 % за 6 с, температура — с 26,0 °C до 26,7 °C. Это свидетельствует о корректной работе датчика и отклике на изменение условий окружающей среды (приближение источника тепла/влаги).

Временная диаграмма для другого значения

В качестве самостоятельно сформированной диаграммы построим передачу значения **RH = 55 %, T = 23,5 °C**. Соответствующие байты:

Поле	Dec	HEX	Биты (MSB→LSB)	Проверка
RH int	55	`0x37`	`00110111`	
RH dec	0	`0x00`	`00000000`	
T int	23	`0x17`	`00010111`	
T dec	5	`0x05`	`00000101`	
CRC	83	`0x53`	`01010011`	$55 + 0 + 23 + 5 = 83$

Table 5: Формирование кадра для значения RH = 55 %, T = 23,5 °C

Итоговая последовательность 40 бит:

```
00110111 00000000 00010111 00000101 01010011
```

Временная диаграмма этой передачи строится по правилу: каждый бит = `LOW` ≈ 54 мкс + `HIGH` ≈ 28 мкс (для “0”) или ≈ 75 мкс (для “1”).

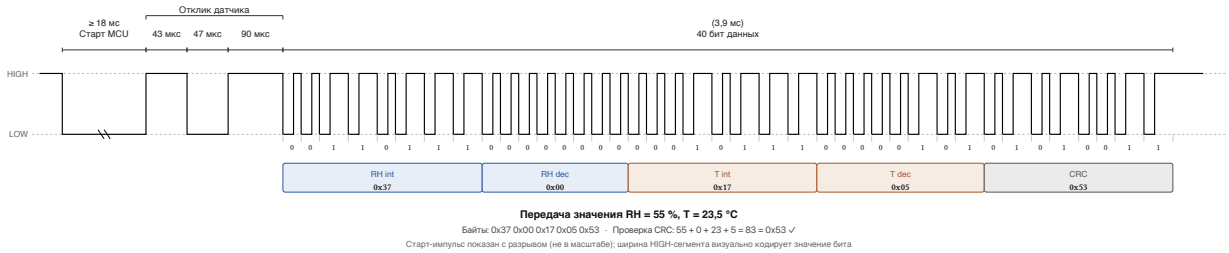

Figure 2: Временная диаграмма передачи RH = 55 %, T = 23,5 °C

Оценка скорости интерфейса

Интерфейс DHT-11 не нормирует фиксированную битовую скорость: длительность каждого бита зависит от его значения, поэтому говорить о скорости в классическом смысле нельзя. Приведём три оценки.

По минимальному периоду бита (бит “0”): период равен $54,3 + 28,3 = 82,6$ мкс, что соответствует мгновенной скорости

$$v_{\max} = \frac{1}{82,6 \cdot 10^{-6}} \approx 12,1 \text{ кбит/с}$$

По участку передачи 40 бит данных: длительность передачи без стартовой последовательности составляет 3,79 мс, тогда

$$v_{\text{data}} = \frac{40}{3,79 \cdot 10^{-3}} \approx 10,6 \text{ кбит/с}$$

По полной транзакции (включая стартовый импульс 18 мс и отклик датчика): полная длительность ≈ 22,1 мс, эффективная скорость передачи полезных данных

$$v_{\text{эфф}} = \frac{40}{22,1 \cdot 10^{-3}} \approx 1,8 \text{ кбит/с}$$

Видно, что пропускная способность интерфейса ограничена в первую очередь обязательной задержкой старт-импульса и отклика датчика. На саму передачу данных тратится менее 20 % времени транзакции, что объясняет низкую эффективную скорость при высокой мгновенной.

Выводы по части DHT-11

1. Зафиксирован и расшифрован обмен по однопроводному протоколу DHT-11: 4 транзакции, во всех CRC сошёлся.
2. Измеренные длительности импульсов соответствуют требованиям даташита.
3. Датчик корректно отрабатывает изменение условий среды (показания влажности и температуры плавно растут от пакета к пакету).
4. Эффективная скорость передачи составляет около 1,8 кбит/с, пиковая — до 12 кбит/с; протокол ориентирован на низкоскоростные измерения, что согласуется с минимальным периодом опроса ≥ 1 с по даташиту.